

Enterprise AI Platform for Scam, Mule Network & Suspicious Transaction Detection

Use Case #47 — Stage 3 Evaluation, TechCon Hackathon, Coforge

Team Qudit Creons

Abhishek Raj | Divyansh Singh

Coforge — #CoforgeTechCon2026 #CoforgeHackathon

Stage 3 Evaluation — 1–2 September 2026

Vision & The Problem

Vision Statement

Give fraud investigators an explainable, real-time view of mule networks and scam patterns — so suspicious money movement is stopped *before cash-out*, not reconstructed after the loss.

Why network-first, not rule-per-transaction: Fraud losses are driven by *networks* of mule accounts moving funds in coordinated patterns — fan-out smurfing, fan-in aggregation, layering chains, rapid pass-through. Row-level rules miss the shape; investigators can't trace it manually at scale.

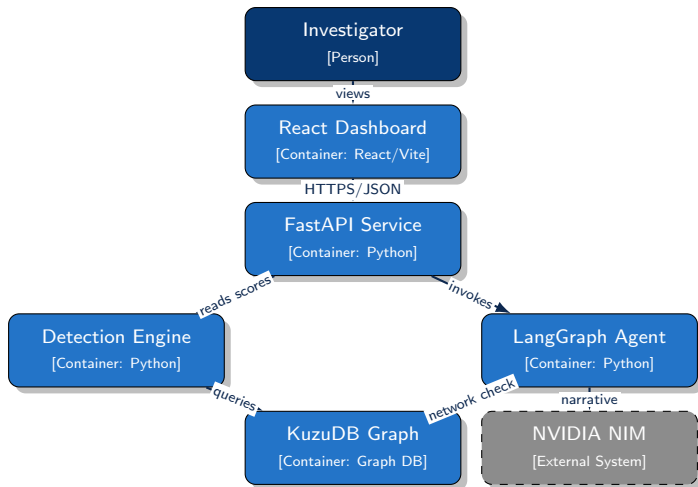
Direct users

- **Investigators** — ranked queue, explainable evidence, network context
- **Compliance officers** — auditable reasoning, SAR-ready typology language
- **Bank customers** — fraud stopped fast; legitimate activity not disrupted

Governance & infra

- **Regulators** — demonstrable due diligence, audit trail by construction
- **Bank IT/Security** — open-source stack, API-first, no core-banking rebuild
- **Leadership** — quantifiable precision gain = investigator-hours saved

Architecture — C4 Container Diagram



C4 model, Container level. Solid = internal container, dashed = external system, dark-blue = person actor.
Full stack open-source except the NIM inference call.

Key Design Decisions

Rule-based, not GNN Every flag explainable by construction — no post-hoc SHAP/GNNExplainer needed to justify a regulatory decision; no training data required.

KuzuDB, not Neo4j Embedded, no JVM overhead — matters on constrained dev/cloud hardware; team had zero prior Cypher experience either way.

Relative thresholds, not fixed Self-calibrated per account. *Proven necessary*: our fixed-threshold detector hit 100% false positives against Santander's independent gen-fraud-graph benchmark — exposed real overfitting to our own dataset's shape.

Damped corroboration scoring Strongest single signal counts fully; weak coincidental signals don't stack to a false flag. Validated by injecting legitimate payroll/marketplace accounts and testing directly against them.

Validation Methodology — Not Just a Demo

- **Adversarial self-testing:** injected legitimate high-volume businesses (payroll processor, marketplace) into our own dataset and measured false positives on them directly — found and fixed a real toy-model gap
- **External benchmark validation:** ran our detectors, unmodified, against Santander AI Lab's independent open-source synthetic fraud graph — surfaced a genuine generalization failure before it could surface in production
- **Iterative, measured improvement:** every scoring change verified against the full evaluation suite before being kept; one regression caught and reverted the same session it was introduced

Result

Precision: 0.746 → 0.855 — Recall: 1.000 (held throughout) — False Positives: 222
→ 110

Live system — <http://80.225.235.192:4173>

- Deployed on Oracle Cloud (Ampere A1.Flex, 4 OCPU / 24GB, Always Free tier)
- Running as managed systemd services — survives reboots, not a laptop demo
- Full pipeline live: flagged accounts → live KuzuDB drill-down → Nemotron case narrative → network-aware escalation

Business Value & Roadmap

Business value

- Fewer false positives = fewer wasted investigator-hours
- Zero missed fraud (recall 1.000) throughout every iteration
- Explainable-by-design = lower regulatory/audit risk than black-box ML
- Fully open-source stack = no vendor lock-in

Next steps

- Continuous scoring for remaining two detectors (fan-out, layering)
- True incremental/live ingestion (currently batch-refresh)
- Formal held-out validation split (current eval is same-dataset)
- Broader external benchmark coverage

Thank You

Q&A

Team Qudit Creons — Abhishek Raj, Divyansh Singh
github.com/Qyngshuk08/145-qudit-creons